

16-Python Dictionary Methods and Creation

Introduction

- The session covers Python dictionary creation, different instantiation methods, common functions, core dictionary methods, error handling, and related programming assignments.
- Includes active Q&A between instructor (Abhishek) and students throughout.

Dictionary Creation Methods

Direct Instantiation

- Dictionary can be created with curly braces `{}` by specifying key-value pairs directly.
- Duplicate keys: Only the last value assigned to a duplicated key is kept (previous values are overwritten).
- Keys can be of any immutable type (number, string, boolean, etc.). Boolean `True` and integer `1` are treated as equal keys; last assignment takes precedence.

Using dict() Function

- The `dict()` function takes an iterable (e.g., list/tuple of pairs) where each element is a (key, value) pair.
- It can accept:
 - List of tuples

- Tuple of tuples
- Tuple of lists
- List of lists
- All pairs must have exactly two elements; otherwise, an error occurs.
- Can store nested data structures (e.g., values as lists/dictionaries).
- String quotations are required for any key/value that is not an identifier/variable.

Handling Data Types in Keys/Values

- Boolean and integer values like `True` and `1` are considered the same key in Python dictionaries.
- String values need quotations; otherwise, Python treats them as undefined identifiers and throws errors.
- Values stored can be any type, e.g., string, int, list, or nested dictionary.

Special Functions for Dictionary Creation

Using `enumerate()`

- `enumerate(iterable)` pairs each element with an index, returns an enumerate object.
- Can be converted to list/tuple:

- `dict(enumerate(my_str))` creates a dictionary with indices as keys and characters as values.
- Index is zero-based by default; can be changed by specifying the `start` argument.
- Useful for associating sequential numbers with data from lists/strings.

Using zip()

- `zip(iter1, iter2, ...)`
combines elements from each iterable by index, returning a zip object (as tuples).
- `dict(zip(keys, values))` associates elements; the shortest iterable determines final length.
- Cannot use zip with more than two iterables in dict(); will cause an error.

Dictionary Comprehension

- Syntax: `{key_exp: value_exp for element in iterable if condition}`
- Quickly generate custom dictionaries using expressions for keys/values.
- Example: `{x*2: x**2 for x in range(1,6)}` creates `{2:1, 4:4, 6:9, 8:16, 10:25}`

Dictionary Methods Overview

Removing Elements

- `.clear()`: Removes all elements from dictionary, leaves an empty dictionary. Returns `None`.

- `.pop(key, [default])`
: Removes and returns the value for a given key; if key not found and default not provided, raises `KeyError`; if default provided, returns default instead.
- `.popitem()`: Removes and returns the last key-value pair as a tuple. No arguments.

Accessing Keys, Values, Items

- `.keys()`
: Returns a non-sequence view of dictionary keys (convert to list/tuple for sequence behavior).
- `.values()`: Same as keys but for values.
- `.items()`: Returns pairs as tuples (non-sequence view).
- Iterating or converting with `list()` provides usable sequences.

Retrieval and Defaults

- `dict[key]`: Returns value for the key, raises `KeyError` if absent.
- `.get(key, [default])`
: Returns value for key; if absent returns `None` or specified default; does not update dictionary.
- `.setdefault(key, [default])`
: Like `get`, but if key absent, inserts it with value `None` or specified default, modifying dictionary.

Updating Dictionaries

- `.update(other_dict)`
: Merges another dictionary into the current one; existing keys are overwritten.

Generating with fromkeys

- `.fromkeys(seq, [default])`
: Creates a new dictionary from keys in sequence, with all values set as default.

Copying Dictionaries

- `.copy()` : Shallow copy; nested objects are still references.
- `deepcopy()` : (from `copy` module): Deep copy duplicates all nested objects.
- Shallow copy: changes to nested structures reflect in both copies; deep copy keeps them independent.

Shallow vs Deep Copy

- Shallow copy duplicates the dictionary object but shares references for nested objects—modifying a nested object affects both copies.
- Deep copy duplicates all objects recursively, so changes are isolated.

Assignments, Q&A and Miscellaneous

- Instructor assigns practical problems using comprehension, enumerate, and zip for practice.
- Clarifications around assignment submission: difference between Jupyter Notebook and Google Colab handling of `.ipynb` files.

- Questions about string assignment repetition, error handling in Python dictionaries, use of dictionary methods, and default values.
- Discussion of methods like `rjust`, `ljust`, and `replace` in string assignments (to be discussed in future sessions).

Class Logistics

- Instructor mentions resource locations (LearnBay content folder, Python folder), assignment schedules, and class times.
 - Notices about holidays and next meeting.
-

Notes powered by [CraftNote](#)